DevSecOps Engineer Home Test

Name :

Date :

1. Scenario

Your company operates a backend API on AWS. The engineering team deploys applications through a CI/CD pipeline, but security controls are inconsistent.

The current environment has the following concerns:

- IAM permissions may be broader than necessary.
- Application secrets could accidentally be committed to Git.
- Docker images are not consistently scanned for vulnerabilities.
- Terraform changes are deployed without adequate security validation.
- Application code is not consistently checked for vulnerabilities.
- The pipeline can continue even when critical security findings exist.
- Production deployment credentials may be stored directly in CI/CD variables.

You are assigned to improve the deployment process by implementing security controls that detect vulnerabilities early and prevent unsafe releases.

Expected outcome :

Design and implement a DevSecOps workflow that uses AWS security best practices and automatically blocks deployments when defined security policies are violated.

a. Task 1 : AWS Architecture and Security Design

Design a secure AWS architecture for deploying a containerized backend API.

Your proposed architecture should include the relevant services from the following list:

- Amazon VPC, public/private subnets, and security groups
- Application Load Balancer or Network Load Balancer
- Amazon ECS with AWS Fargate, or Amazon EKS
- Amazon ECR for container image storage
- AWS IAM roles and policies
- AWS Secrets Manager or Systems Manager Parameter Store
- Amazon CloudWatch for logging and monitoring
- AWS CloudTrail for audit trails
- AWS WAF where appropriate
- AWS KMS for encryption where appropriate

Requirements

1. Explain the request flow from the internet to the application.
2. Identify which resources should be public and which should be private.
3. Explain how the application retrieves secrets without hardcoding them.
4. Describe the IAM roles required by the CI/CD pipeline and the runtime application.
5. Explain how you would prevent unauthorized access to production resources.
6. Identify at least five security risks and the corresponding mitigations.

DevSecOps Engineer Home Test

Deliverables

- Architecture diagram.
- Short explanation of the security controls.
- Example IAM policy or Terraform configuration for one critical control.

You do not need to implement every AWS service. Prioritize the controls that materially reduce security risk.

b. Task 2 : CI/CD Security Pipeline

Create a CI/CD pipeline for a sample backend application using GitLab CI/CD, GitHub Actions, or Jenkins.

The pipeline must demonstrate the following stages:

1. Source validation.
2. SAST : Static Application Security Testing.
3. SCA : Software Composition Analysis.
4. Secret scanning.
5. Terraform/IaC security scanning.
6. Docker image build.
7. Container vulnerability scanning.
8. Deployment approval or deployment simulation.

Required security tools

Use the following tools or reasonable alternatives:

Security control	Suggested tool
SAST	SonarQube
SCA / dependency scanning	Trivy, OWASP Dependency-Check, or equivalent
Secret scanning	Gitleaks
IaC scanning	Checkov or Trivy
Container scanning	Trivy
DAST	OWASP ZAP

A single tool may support multiple controls, provided you explain which checks are performed and how their results are evaluated.

Pipeline requirements

- Security scanning runs automatically on code changes.
- Pipeline artifacts or logs provide evidence of scan results.
- Critical findings trigger a failed pipeline.
- Lower-severity findings are handled according to a documented policy.
- Production deployment requires an appropriate approval or protected deployment mechanism.
- Secrets must not be printed in logs or embedded in the repository.
- Pipeline jobs should use least-privilege credentials.

Deliverables

- Working pipeline configuration.
- Example scan output or execution evidence.
- Explanation of the security gates.

- A short description of how false positives and accepted risks are handled.

c. Task 3 : Terraform and AWS IAM Security

Create Terraform configuration for a minimal AWS deployment, or improve a provided example if you prefer to use your own repository.

The configuration should demonstrate:

- An AWS provider and appropriately scoped infrastructure resources.
- IAM least-privilege principles.
- Encryption at rest where supported.
- Restricted network access.
- Secure handling of application secrets.
- Resource tagging and environment separation.
- Automated IaC security scanning.

Mandatory security scenarios

Scenario A : Overly permissive IAM

A developer proposes the following policy for an application deployment role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

Explain why this policy is dangerous and propose a safer approach.

Your solution should consider:

- Resource-level permissions where supported.
- Separate deployment and application runtime roles.
- Short-lived credentials.
- Cross-account access, if applicable.

Scenario B : Publicly accessible resource

A Terraform change creates an application database that is accessible from 0.0.0.0/0.

Explain how your pipeline detects and blocks this configuration before deployment.

Scenario C : Secret exposure

An application password is stored in Terraform variables and appears in a state file.

Explain how you would prevent further exposure, secure the secret, and handle the existing exposed credential and Terraform state.

DevSecOps Engineer Home Test

Deliverables

- Terraform files.
- Example IAM policy or role configuration.
- IaC scan output.
- Brief remediation notes for all three scenarios.

d. Task 4 : Container and Application Security

Create a Dockerfile for a small backend application or use an existing sample application.

Demonstrate the following:

- Use of a minimal, trusted base image.
- Running the application as a non-root user where feasible.
- Avoidance of unnecessary packages and tools.
- Dependency version control.
- Container vulnerability scanning.
- Secure configuration through environment variables or a secrets-management mechanism.
- No secrets embedded in Docker image layers.

Vulnerability management requirements

Document a vulnerability policy with severity-based handling.

For example:

Finding	Expected handling
Critical vulnerability	Block release by default
High vulnerability	Block release according to policy and exposure
Medium vulnerability	Track remediation and assess risk
Low vulnerability	Track and remediate according to risk

Explain how you would handle a critical vulnerability when no patched version is available.

Deliverables

- Dockerfile.
- Container scan results.
- Short vulnerability remediation policy.

e. Task 5 : Troubleshooting and Incident Response

For each scenario, describe your investigation steps, likely root causes, and corrective actions.

Scenario A : Pipeline security failure

A pipeline fails because Trivy reports a critical vulnerability in a base image.

The development team says the application is working correctly and requests an immediate production deployment.

What would you do?

Your response should address risk assessment, remediation, exceptions, and release authorization.

DevSecOps Engineer Home Test

Scenario B : ECS application cannot retrieve secrets

An ECS Fargate task fails to start because it cannot retrieve a secret from AWS Secrets Manager.

Identify the AWS components and permissions you would investigate, including:

- ECS task execution role versus task role.
- Secrets Manager resource and permissions.
- KMS key permissions, if a customer-managed key is used.
- Network connectivity and relevant VPC endpoints or outbound access.
- CloudWatch logs and ECS task events.

Explain how you would isolate the problem without granting broad administrator permissions.

Scenario C : Unauthorized AWS API activity

CloudTrail shows unexpected API calls from an IAM principal used by the deployment pipeline.

Describe your immediate containment actions, investigation process, and recovery steps.

Consider credential revocation, affected resources, audit evidence, and the possibility that credentials or permissions have been compromised.

Deliverables

Submit concise written answers for all three scenarios.

f. Task 6 : DAST and OWASP Top 10

Demonstrate how you would incorporate DAST into the pipeline using OWASP ZAP or an equivalent tool.

Explain:

1. How the application is made available in a safe test environment.
2. Which DAST checks are executed automatically.
3. How authentication is handled during testing.
4. Which findings should block a release.
5. How you prevent security testing from affecting production systems.

Identify at least three relevant OWASP Top 10 risks and describe how you would detect or mitigate them.

> *Use only applications and environments that you own or are explicitly authorized to test.*

g. Submission Requirements

Submit a Git repository containing the following structure, or a logically equivalent structure:

```
devsecops-home-test/
 ├── README.md
 ├── architecture/
|   └── architecture-diagram.png
 ├── app/
|   ├── Dockerfile
|   └── src/
 ├── terraform/
|   ├── main.tf
|   ├── variables.tf
|   └── outputs.tf
 ├── .gitlab-ci.yml
```

```
|   # or .github/workflows/ / Jenkinsfile
├── security/
|   ├── vulnerability-policy.md
|   └── incident-response.md
└── evidence/
    └── scan-results.md
```

The directory names are illustrative. Candidates may adapt them to their chosen tools and application.

README must include

- Architecture overview.
- Prerequisites and execution instructions.
- CI/CD workflow and security gates.
- Security decisions and assumptions.
- Example commands to validate Terraform and scan the container.
- Known limitations and remaining risks.
- Evidence of tests and scans performed.

> ***Security requirements:*** *Do not submit AWS access keys, passwords, tokens, production data, or other sensitive information. Redact sensitive values from screenshots and logs.*